

(b) Consider a network comprising five routers labeled as A, B, C, D, and E. The interconnections and their associated costs are given as follows :

- Link between A and B: Cost 2
- Link between A and C: Cost 4
- Link between B and C: Cost 1
- Link between B and D: Cost 5
- Link between C and D: Cost 3
- Link between C and E: Cost 7
- Link between D and E: Cost 2

कालिन्दी महाविद्यालय पुस्तकालय
KALINDI COLLEGE LIBRARY

4133

5

Using Dijkstra's algorithm, illustrate the step-by-step process of determining the shortest path from router A to router E. (8)

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question requires the use of **Dijkstra's Algorithm**. It is a foundational concept in graph theory and networking, frequently applied in dynamic routing scenarios—such as routing data packets across the internet or managing node communication in Vehicular Ad Hoc Networks (VANETs)—to find the most efficient, lowest-cost path between a starting node and all other nodes in a graph.

Here is a breakdown of the concept, the steps to solve it, and the demonstration using your specific problem.

The Concept

Dijkstra's Algorithm is a "greedy" algorithm. It works by progressively exploring the network starting from the source node. At each step, it looks at the unvisited node closest to the start, examines its direct neighbors, and calculates if going through this current node offers a shorter path to those neighbors than previously known. It locks in the shortest distance one node at a time until the destination is reached.

Steps to Solve

To solve these problems by hand, it is best to use a tabular method to track the shortest known distance from the starting node to every other node.

1. **Initialization:** * Set the distance to the starting node to 0.
 - Set the distance to all other nodes to infinity (∞).
 - Create a list of all nodes and mark them as "Unvisited."
2. **Node Selection:** * From the "Unvisited" list, select the node with the smallest known distance. Let's call this the *Current Node*.
3. **Relaxation (Updating Neighbors):** * Look at all unvisited neighbors of the Current Node.

- Calculate the tentative distance to each neighbor: **Distance to Current Node** + **Cost of Edge to Neighbor** .
 - If this tentative distance is *less* than the neighbor's currently recorded distance, update the neighbor's distance to this new, lower value and record the Current Node as its "predecessor" (how it got there).
4. **Mark as Visited:** * Once all neighbors of the Current Node have been evaluated, remove the Current Node from the "Unvisited" list. Its shortest distance is now finalized.
5. **Repeat:** * Repeat steps 2–4 until the destination node is marked as visited, or the "Unvisited" list is empty.

Demonstration: Path from Router A to E

Based on your image, we have the following network edges and costs:

- (A, B) = 2, (A, C) = 4
- (B, C) = 1, (B, D) = 5
- (C, D) = 3, (C, E) = 7
- (D, E) = 2

We will track the distances in a table. The letter in parentheses next to a distance represents the predecessor node (which router we came from to get that distance).

Step 0: Initialization

- Set A to 0, everything else to ∞ .

Step	Selected Node	Dist to A	Dist to B	Dist to C	Dist to D	Dist to E	Unvisited Nodes
0	-	0	∞	∞	∞	∞	A, B, C, D, E

Step 1: Evaluate A

- Smallest unvisited is **A** (0).
- Neighbors of A are B (cost 2) and C (cost 4).
- Path to B: $0 + 2 = 2$. ($2 < \infty$, so update B).
- Path to C: $0 + 4 = 4$. ($4 < \infty$, so update C).
- Mark A as visited.

Step	Selected Node	Dist to A	Dist to B	Dist to C	Dist to D	Dist to E	Unvisited Nodes
1	A	0	2 (A)	4 (A)	∞	∞	B, C, D, E

Step 2: Evaluate B

- Smallest unvisited is **B** (2).
- Unvisited neighbors of B are C (cost 1) and D (cost 5).
- Path to C via B: $2 + 1 = 3$. ($3 < 4$, so update C to 3).
- Path to D via B: $2 + 5 = 7$. ($7 < \infty$, so update D).
- Mark B as visited.

Step	Selected Node	Dist to A	Dist to B	Dist to C	Dist to D	Dist to E	Unvisited Nodes
2	B	0	2	3 (B)	7 (B)	∞	C, D, E

Step 3: Evaluate C

- Smallest unvisited is **C** (3).
- Unvisited neighbors of C are D (cost 3) and E (cost 7).
- Path to D via C: $3 + 3 = 6$. ($6 < 7$, so update D to 6).
- Path to E via C: $3 + 7 = 10$. ($10 < \infty$, so update E).
- Mark C as visited.

Step	Selected Node	Dist to A	Dist to B	Dist to C	Dist to D	Dist to E	Unvisited Nodes
3	C	0	2	3	6 (C)	10 (C)	D, E

Step 4: Evaluate D

- Smallest unvisited is **D** (6).
- Unvisited neighbors of D is E (cost 2).
- Path to E via D: $6 + 2 = 8$. ($8 < 10$, so update E to 8).
- Mark D as visited.

Step	Selected Node	Dist to A	Dist to B	Dist to C	Dist to D	Dist to E	Unvisited Nodes
4	D	0	2	3	6	8 (D)	E

Step 5: Evaluate E

- Smallest unvisited is **E** (8). It has no unvisited neighbors. Mark as visited. Algorithm completes.

Final Answer

The shortest total cost from Router A to Router E is **8**. To find the actual path, trace backwards using the predecessors from the table:

- E came from D
- D came from C
- C came from B
- B came from A

The Shortest Path is: **A → B → C → D → E**

Here is an interactive visualization of the network provided in your question so you can trace how the algorithm traverses the edges.

```
json
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Objective: Build an interactive graph visualization of Dijkstra's Algorithm allowing the user to step through the shortest path calculation for a specific network. \n\nData State: \nNodes: A, B, C, D, E.\nEdges with weights: A-B (2), A-C (4), B-C (1), B-D (5), C-D (3), C-E (7), D-E (2). \nSource Node: A. Target Node: E.\n\nStrategy: Standard Layout using D3.js or a similar graphing library.\n\nInputs: \n1. A 'Next Step' button to advance the algorithm one iteration at a time.\n2. A 'Reset' button to clear the state.\n\nBehavior: \nRender the 5 nodes visually connected by edges, with the edge weights clearly labeled on the connecting lines. Alongside the graph, display a dynamic Distance Table tracking the shortest known distance to each node (initializing A to 0 and others to infinity). \n\nWhen the user clicks 'Next Step':\n1. Visually distinguish the 'Current Node' being evaluated.\n2. Briefly highlight the edges being checked to its neighbors.\n3."
  }
}
```

```
Update the Distance Table if a shorter path is found.\n4. Distinctly mark nodes as 'Visited' once their evaluation is complete.\n\nOnce the algorithm reaches node E, strongly highlight the final shortest path edges (A-B, B-C, C-D, D-E) and display a success message stating 'Shortest Path: A -> B -> C -> D -> E (Total Cost: 8)'.", "id": "im_94b8498d8745aea3"}}
```

You Asked 

3. (a) Consider a Go-Back-N ARQ protocol operating over a network. The window size (W) is 4, and the sequence numbers range from 0 to 7. Assume that frames with sequence numbers 0, 1, 2, and 3 have been successfully received by the receiver, and the receiver has sent acknowledgment (ACK) for frames up to sequence number 3. Now, due to network congestion, the frame with sequence number 4 is lost. (7)

- (i) What is the size of the window for sender and receiver?
- (ii) Calculate the range of valid sequence numbers at the sender's side after the loss of frame with sequence number 4.
- (iii) Determine the range of sequence numbers expected by the receiver after it has sent ACK for frames up to sequence number 3.
- (iv) Explain how the sender and receiver handle the loss of frame 4 and maintain synchronization using sequence numbers in the Go-Back-N ARQ protocol.

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question explores the **Go-Back-N (GBN) Automatic Repeat reQuest (ARQ)** protocol. This is a foundational concept in computer networking used to ensure reliable data transmission over unreliable channels. Understanding how these protocols handle packet loss is crucial when researching or designing networks that require high reliability, such as Vehicular Ad Hoc Networks (VANETs) or the Internet of Medical Things (IoMT), where missing data can cause significant system disruptions.

Here is a breakdown of the concept, the general steps to solve these problems, and a detailed solution to the specific questions posed in the image.

The Concept: Go-Back-N ARQ

Go-Back-N is a specific instance of a "sliding window" protocol. It allows a sender to transmit multiple frames (up to a specified window size, W) before requiring an acknowledgment (ACK) from the receiver. This keeps the network pipeline full and efficient.

- **The Sender:** Maintains a "window" of frames it is allowed to send. Once it sends a frame, it starts a timer. If it receives an ACK, the window slides forward, allowing new frames to be sent.
- **The Receiver:** In GBN, the receiver is very strict. It *only* accepts frames in the exact sequential order. Its window size is always exactly 1.

- **Handling Errors ("Going Back N"):** If a frame is lost in transit (like frame 4 in this problem), the receiver will never receive it. If the sender continues to send frames 5, 6, and 7, the receiver will simply discard them because it is still stubbornly waiting for frame 4. Eventually, the sender's timer for frame 4 will expire. The sender must then "go back" to the unacknowledged frame (frame 4) and retransmit it, along with all subsequent frames in its current window (5, 6, and 7).

General Steps to Solve Sliding Window Problems

To systematically solve ARQ protocol questions, follow these steps:

1. **Identify Protocol Parameters:** Determine the protocol type (Go-Back-N vs. Selective Repeat), the sequence number range (e.g., 0 to 7), and the sender window size (W).
2. **Determine Window Sizes:** Remember that for GBN, the sender window size (W_s) is N (given in the problem), but the receiver window size (W_r) is always strictly 1.
3. **Track the Current State:** Map out which frames have been sent, which have been successfully acknowledged, and where the sender's window currently sits.
4. **Apply Error Rules:** If a frame is lost, determine how the specific protocol reacts. In GBN, note that all frames sent after a lost frame will be discarded by the receiver, forcing a full window retransmission upon timeout.

Demonstration: Solving the Question

Given Data:

- Protocol: Go-Back-N ARQ
- Window Size (W) = 4
- Sequence Numbers = 0 to 7
- Current State: Frames 0, 1, 2, 3 successfully received and ACKed. Frame 4 is lost.

(i) What is the size of the window for sender and receiver?

- **Sender Window Size (W_s):** The problem states the window size W is 4. Therefore, the sender can transmit up to 4 unacknowledged frames at a time. $W_s = 4$.
- **Receiver Window Size (W_r):** In the Go-Back-N protocol, the receiver only ever looks for one specific frame at a time (the next one in sequence). It does not buffer out-of-order frames. Therefore, $W_r = 1$.

(ii) Calculate the range of valid sequence numbers at the sender's side after the loss of frame with sequence number 4.

Because frames 0, 1, 2, and 3 have been successfully acknowledged, the sender's window has slid past them. The sender is now authorized to send the next 4 frames in the sequence.

- The newly available frames are 4, 5, 6, and 7.
- **Answer:** The valid range of sequence numbers currently in the sender's window is **[4, 7]** (or 4, 5, 6, 7).

(iii) Determine the range of sequence numbers expected by the receiver after it has sent ACK for frames up to sequence number 3.

Because the receiver's window size is exactly 1, it does not expect a "range" of multiple numbers. Having successfully received frame 3, its window slides forward by exactly one position.

- **Answer:** The receiver is expecting strictly sequence number 4.

(iv) Explain how the sender and receiver handle the loss of frame 4 and maintain synchronization...

- **The Receiver's Action:** Because frame 4 is lost, the receiver never gets it. If the sender continues to transmit frames 5, 6, and 7, they arrive out of order. Because $W_r = 1$, the receiver silently discards frames 5, 6, and 7. It may optionally send

duplicate ACKs for frame 3 to indicate it is still waiting for 4.

- **The Sender's Action:** The sender transmits frames 4, 5, 6, and 7. It waits for an ACK for frame 4, which never comes. Eventually, the timeout timer for frame 4 expires.
- **Maintaining Synchronization (Recovery):** Upon timeout, the sender realizes frame 4 was lost. Following the Go-Back-N rule, it resets its transmission state and **retransmits the entire current window**: frames 4, 5, 6, and 7. The receiver, still waiting for frame 4, successfully receives it this time, accepts it, and synchronization is restored.

To help visualize how the windows slide and how the system recovers from this exact frame loss scenario, you can explore the interactive widget below.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Objective: Build an interactive visualizer for the Go-Back-N ARQ protocol demonstrating window sliding and frame loss recovery based on the specific problem state. \n\nData State: \nSequence Numbers: [0, 1, 2, 3, 4, 5, 6, 7]. \nSender Window Size (Ws) = 4. \nReceiver Window Size (Wr) = 1. \nInitial Condition: Frames 0, 1, 2, 3 are already marked as Sent and Acknowledged. The Sender's current active window covers frames [4, 5, 6, 7]. The Receiver's active window covers frame [4].\n\nStrategy: Standard Layout. Display the Sender's sequence numbers horizontally at the top, and the Receiver's sequence numbers horizontally at the bottom. Use visual boxes to represent the 'sliding window' encompassing the active frames on both sides.\n\nInputs: \n1. 'Send Next Frame' button.\n2. 'Simulate Frame Loss' button.\n3. 'Trigger Timeout (Retransmit)' button.\n4. 'Reset' button.\n\nBehavior: \n- Render the sender and receiver timelines. Clearly highlight the Sender Window (covering 4 frames) and Receiver Window (covering 1 frame). \n- Clicking 'Send Next Frame' should animate a packet moving from the sender's current active frame down to the receiver. If received in order, slide the receiver window forward, animate an ACK returning, and slide the sender window forward.\n- Clicking 'Simulate Frame Loss' (when on frame 4) should animate the packet moving but disappearing midway. If the user then sends subsequent frames (5, 6, 7), animate them reaching the receiver but visually show them being 'Discarded' (bouncing off or turning red) because the receiver window is stuck on 4.\n- Clicking 'Trigger Timeout' should instantly reset the sender's transmission pointer back to the start of its current window (frame 4) and allow the user to resend 4, 5, 6, 7 successfully.",
    "id": "im_67d493628eff2e72"
  }
}
```

You Asked 

(b) Given the IP address 192.168.0.0/24, an organization needs to create two subnets with the following requirements: (10)

- Subnet 1: 50 hosts
- Subnet 2: 30 hosts

For each of these subnets, give the

- (i) first IP address assigned
- (ii) last IP address assigned
- (iii) subnet mask in the w.x.y.z/s notation
- (iv) identify the range of IP addresses available
- (v) number of host and network bits

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

This question is based on the concept of **Subnetting**, specifically utilizing **Variable Length Subnet Masking (VLSM)**. Subnetting is the logical practice of dividing a single, large network into multiple smaller, more efficient networks (subnets).

When designing networks, routers broadcast traffic to all hosts within a subnet. If a subnet is too large, it creates unnecessary broadcast traffic and wastes IP addresses. VLSM allows network administrators to allocate IP address blocks that perfectly match the required number of hosts for each specific department or segment, maximizing the efficiency of the available IP space.

Steps to Solve Subnetting Problems

To accurately divide a network and find the parameters for each subnet, follow these sequential steps:

1. **Sort the Requirements:** Always allocate IP addresses starting with the subnet that requires the *largest* number of hosts, working down to the smallest.
2. **Determine Host Bits (h):** Use the formula $2^h - 2 \geq \text{Required Hosts}$ to find the minimum number of host bits needed. We subtract 2 because the first address in any block is reserved for the Network ID, and the last is reserved for the Broadcast Address.
3. **Determine Network Bits (n):** An IPv4 address is always 32 bits. Subtract your host bits from 32 to find the network bits ($32 - h = n$). This gives you your CIDR notation ($/n$).
4. **Calculate the Subnet Mask:** Convert the n network bits into 1s and the h host bits into 0s to get the binary subnet mask, then translate it into w.x.y.z dotted-decimal format.
5. **Find the Block Size:** The total number of IPs in the subnet (including network and broadcast) is 2^h .
6. **Define Address Ranges:**
 - * **Network Address:** The starting boundary of the block.
 - **First Assignable IP:** Network Address + 1.
 - **Broadcast Address:** The last IP of the block (or the next subnet's Network Address - 1).
 - **Last Assignable IP:** Broadcast Address - 1.

Demonstration: Solving the Question

Given Base Network: 192.168.0.0/24 (Total available IPs = 256) **Requirements sorted by size:** Subnet 1 (50 hosts), Subnet 2 (30 hosts).

Solving for Subnet 1 (50 hosts)

- **Host bits:** $2^h - 2 \geq 50$. The smallest power of 2 that works is 6 ($2^6 = 64$; $64 - 2 = 62$ usable hosts). So, $h = 6$.
- **Network bits:** $32 - 6 = 26$. This makes the notation **/26**.
- **Subnet Mask:** 26 network bits (1s) and 6 host bits (0s) equates to **255.255.255.192**.
- **Block Size:** $2^6 = 64$.
- **Network Address:** 192.168.0.0.
- **Broadcast Address:** 192.168.0.63.

Solving for Subnet 2 (30 hosts)

- **Starting Point:** The next available IP block begins immediately after Subnet 1 ends, which is **192.168.0.64**.
- **Host bits:** $2^h - 2 \geq 30$. The smallest power of 2 that works is 5 ($2^5 = 32$; $32 - 2 = 30$ usable hosts). So, $h = 5$.

- **Network bits:** $32 - 5 = 27$. This makes the notation **/27**.
- **Subnet Mask:** 27 network bits (1s) and 5 host bits (0s) equates to **255.255.255.224**.
- **Block Size:** $2^5 = 32$.
- **Network Address:** 192.168.0.64.
- **Broadcast Address:** $192.168.0.64 + 31 = 192.168.0.95$.

Final Answers Overview

Here are the specific answers to items (i) through (v) for both subnets, derived from the calculations above:

Requirement	Subnet 1 (50 Hosts)	Subnet 2 (30 Hosts)
(i) First IP assigned	192.168.0.1	192.168.0.65
(ii) Last IP assigned	192.168.0.62	192.168.0.94
(iii) Subnet mask (w.x.y.z/s)	255.255.255.192/26	255.255.255.224/27
(iv) Range of IP addresses	192.168.0.1 – 192.168.0.62	192.168.0.65 – 192.168.0.94
(v) Host and Network bits	Host = 6, Network = 26	Host = 5, Network = 27

To better understand how different host requirements change the subnet mask and eat into a /24 block of IP addresses, you can experiment with the interactive VLSM calculator below.

```
json
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Objective: Build an interactive VLSM (Variable Length Subnet Mask) calculator and visualizer.\n\nData State: \nInitial Base Network: 192.168.0.0. Initial CIDR: 24.\nInitial Subnets: Subnet 1 (50 hosts), Subnet 2 (30 hosts).\n\nStrategy: Form Layout combined with a dynamic data table and visual representation.\n\nInputs: \n1. Input fields for Base IP Address and Base CIDR (defaulting to the Data State).\n2. A dynamic list of 'Required Subnets' where the user can add new subnets, define a custom name, and input the number of required hosts.\n3. A 'Calculate VLSM' button.\n\nBehavior: \nUpon calculation, sort the subnets by required hosts in descending order. Generate a results table displaying for each subnet: Name, Needed Hosts, Allocated Size, Network Address, CIDR, Subnet Mask, First Usable IP, Last Usable IP, and Broadcast IP. \nBelow the table, render a segmented visual bar representing the total Base Network space (e.g., 256 IPs for a /24). Visually divide the bar into proportional blocks representing the allocated subnets, distinctively coloring them to show how much space is used and how much remains unallocated.",
    "id": "im_f7dee192262e4b73"
  }
}
```

You Asked

- (b) (i) A code includes the four four-bit codewords: 1001, 0110, 1010, and 0101.
- What is the minimum distance of this code?
 - What is the maximum number of errors that this code is guaranteed to detect?
 - What is the maximum number of errors that this code is guaranteed to correct?

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question is based on the concept of **Error Detection and Correction** in digital communications, specifically relying on **Hamming Distance**.

When data is transmitted over a network, noise can flip bits (changing a 0 to a 1 or vice-versa), causing errors. To combat this, we group data into specific valid sequences called "codewords." By carefully selecting codewords that are mathematically "far apart" from each other, the receiver can detect if a bit has flipped, and sometimes even determine which bit flipped to correct it.

Here is a breakdown of the core concepts, the steps to solve any problem like this, and the specific solution to your question.

The Concepts

- **Hamming Distance (d):** The number of bit positions in which two codewords differ. For example, 1001 and 1010 differ in the last two positions, so their Hamming distance is 2.
- **Minimum Hamming Distance (d_{min}):** The smallest Hamming distance between any two distinct codewords in the entire codebook. This value dictates the strength of the code.
- **Error Detection Capability:** A code can guarantee the detection of up to s errors as long as the number of errors is less than the minimum distance. The formula is: $MaximumDetectableErrors = d_{min} - 1$
- **Error Correction Capability:** A code can guarantee the correction of up to t errors if the minimum distance is large enough that a corrupted codeword is still strictly closer to its original codeword than to any other valid codeword. The formula is: $MaximumCorrectableErrors = \lfloor \frac{d_{min}-1}{2} \rfloor$ (Note: The brackets $\lfloor . . \rfloor$ mean to round down to the nearest whole number).

Steps to Solve

1. **List the Codewords:** Identify all the valid codewords provided in the problem.
2. **Calculate Pairwise Distances:** Compare every possible pair of codewords. Count how many bits differ between each pair.
3. **Find the Minimum Distance (d_{min}):** Identify the smallest number calculated in Step 2.
4. **Calculate Detection:** Subtract 1 from d_{min} to find the maximum number of errors the code can detect.
5. **Calculate Correction:** Subtract 1 from d_{min} , divide by 2, and round down to find the maximum number of errors the code can correct.

Demonstration: Solving Your Question

Given Codewords:

- $C_1 = 1001$
- $C_2 = 0110$
- $C_3 = 1010$
- $C_4 = 0101$

Step 1 & 2: Calculate Pairwise Distances We must compare every unique combination of these four codewords.

Pair	Comparison	Differing Bit Positions	Hamming Distance
C_1, C_2	1001 vs 0110	1st, 2nd, 3rd, 4th	4
C_1, C_3	1001 vs 1010	3rd, 4th	2
C_1, C_4	1001 vs 0101	1st, 2nd	2
C_2, C_3	0110 vs 1010	1st, 2nd	2
C_2, C_4	0110 vs 0101	3rd, 4th	2
C_3, C_4	1010 vs 0101	1st, 2nd, 3rd, 4th	4

Step 3: Find the Minimum Distance Looking at the results in the table (4, 2, 2, 2, 2, 4), the smallest distance between any two valid codewords is 2.

- **Answer 1:** The minimum distance of this code (d_{min}) is 2.

Step 4: Maximum Number of Errors Detected Using the detection formula: $d_{min} - 1$

- $2 - 1 = 1$
- **Answer 2:** This code is guaranteed to detect a maximum of **1 error**. (If 2 errors occur, for example, 1001 might turn into 1010, which is another valid codeword. The system would wrongly accept it as valid, failing to detect the errors).

Step 5: Maximum Number of Errors Corrected Using the correction formula: $\lfloor \frac{d_{min}-1}{2} \rfloor$

- $\lfloor \frac{2-1}{2} \rfloor = \lfloor \frac{1}{2} \rfloor = 0$
- **Answer 3:** This code is guaranteed to correct a maximum of **0 errors**. (Because the minimum distance is only 2, if 1 bit flips, the resulting invalid code is equally close to two different valid codewords. The receiver knows an error happened, but cannot mathematically determine which original codeword it was supposed to be).

To help you explore how changing the codewords affects a system's ability to detect and correct errors, you can use the interactive widget below.

```
json
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "600px",
    "prompt": "Objective: Build an interactive Hamming Distance and Error Capability calculator for block codes.\n\nData State: \nInitial Codewords: ['1001', '0110', '1010', '0101'].\n\nStrategy: Form Layout with dynamic data visualization.\n\nInputs: \n1. A text input field to type a new binary codeword (e.g., '1111').\n2. An 'Add Codeword' button.\n3. Small 'x' buttons next to each active codeword in a list to remove them.\n\nBehavior: \n- Maintain a list of active codewords. Ensure all codewords must be the same length (validate input).\n- Automatically generate and display a 'Pairwise Distance Table' (or matrix) that cross-references every active codeword against the others to show their Hamming distances.\n- Distinctly highlight the smallest number(s) in this table, representing the Minimum Hamming Distance (d_min).\n- Below the table, display three prominently calculated metrics based on the current list:\n  1. Minimum Distance (d_min)\n  2. Max Detectable Errors (d_min - 1)\n  3. Max Correctable Errors (Floor of (d_min - 1) / 2)\n- If the user adds or removes a codeword, immediately recalculate and update the table and the three metrics.",
    "id": "im_ec7c6bf3b5dcdca"
  }
}
```

(ii) A 12-bit odd-parity Hamming code whose binary value is 111001001111 arrives at a receiver. What was the original value of the message ? Assume that not more than 1 bit is in error. (4+6)

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question requires the use of **Hamming Code Error Correction**, specifically focusing on **Odd Parity**.

Hamming code is a mathematical method used in digital communications to not only detect if data has been corrupted in transit but also to pinpoint exactly which bit flipped, allowing the receiver to automatically correct it. It achieves this by inserting redundant "parity bits" at specific positions alongside the actual data bits.

Here is a breakdown of the core concept, the steps to solve these problems, and a demonstration using your specific exam question.

The Concept

- **Bit Positions:** In a Hamming code, bits are assigned position numbers (1, 2, 3, 4, etc.).
- **Parity Bits vs. Data Bits:** Bits at positions that are powers of 2 (1, 2, 4, 8, etc.) are designated as **Parity Bits** (P_1, P_2, P_4, P_8). All other positions (3, 5, 6, 7, 9, etc.) contain the actual **Data Bits**.
- **Parity Groups:** Each parity bit is responsible for checking a specific alternating sequence of bits in the message.
- **Odd Parity:** This rule dictates that the total count of **1** s within any given parity group (including the parity bit itself) must be an **odd number**. If the receiver counts an even number of **1** s, it knows an error occurred in that group.

Steps to Solve

1. **Assign Positions:** Number the received bits. (Crucial Note: While left-to-right indexing is common, if evaluating left-to-right yields an impossible error position—like position 13 in a 12-bit code—you must evaluate using Right-to-Left indexing, which is another standard convention).
2. **Identify the Groups:** Map out which bit positions belong to which parity checks:
 - P_1 checks positions: 1, 3, 5, 7, 9, 11
 - P_2 checks positions: 2, 3, 6, 7, 10, 11
 - P_4 checks positions: 4, 5, 6, 7, 12
 - P_8 checks positions: 8, 9, 10, 11, 12
3. **Calculate the Syndrome:** For each parity group, count the number of **1** s.
 - If the count is **odd** (Parity passes) $\rightarrow$ Syndrome bit = 0
 - If the count is **even** (Parity fails) $\rightarrow$ Syndrome bit = 1
4. **Find the Error:** Combine the syndrome bits in reverse order ($S_8 S_4 S_2 S_1$) to form a binary number. Convert this to decimal to find the exact position of the incorrect bit.
5. **Correct and Extract:** Flip the bit at the identified position. Finally, drop all the parity bits (at positions 1, 2, 4, and 8) to reveal the original, uncorrupted data message.

Demonstration: Solving Your Question

Received Value: 1 1 1 0 0 1 0 0 1 1 1 1

Analytical Note: If we number these bits from Left-to-Right, the resulting syndrome calculation yields 13. Since there is no position 13 in a 12-bit code, this confirms the string must be evaluated using **Right-to-Left indexing** (Rightmost bit = Position 1).

Step 1: Assign Positions (Right-to-Left)

12	11	10	9	8	7	6	5	4	3	2	1
1	1	1	0	0	1	0	0	1	1	1	1
D_{12}	D_{11}	D_{10}	D_9	P_8	D_7	D_6	D_5	P_4	D_3	P_2	P_1

Step 2 & 3: Check Parity Groups (Targeting Odd Parity)

- Check P_1 (Positions 1, 3, 5, 7, 9, 11):
 - Bits: 1, 1, 0, 1, 0, 1
 - Count of 1s: 4 (Even).
 - Result: Parity Fails $\rightarrow S_1 = 1$
- Check P_2 (Positions 2, 3, 6, 7, 10, 11):
 - Bits: 1, 1, 0, 1, 1, 1
 - Count of 1s: 5 (Odd).
 - Result: Parity Passes $\rightarrow S_2 = 0$
- Check P_4 (Positions 4, 5, 6, 7, 12):
 - Bits: 1, 0, 0, 1, 1
 - Count of 1s: 3 (Odd).
 - Result: Parity Passes $\rightarrow S_4 = 0$
- Check P_8 (Positions 8, 9, 10, 11, 12):
 - Bits: 0, 0, 1, 1, 1
 - Count of 1s: 3 (Odd).
 - Result: Parity Passes $\rightarrow S_8 = 0$

Step 4: Find the Error Position Combine the syndrome bits: $S_8S_4S_2S_1 = 0001_2$ Converting binary 0001 to decimal gives 1. Therefore, the error is at **Position 1**.

Step 5: Correct and Extract

- The bit at Position 1 was received as a 1. We must correct it by flipping it to a 0.
- Corrected 12-bit code: 1 1 1 0 0 1 0 0 1 1 1 0
- To find the original message, extract only the data bits ($D_{12}, D_{11}, D_{10}, D_9, D_7, D_6, D_5, D_3$):
- Original Message: 1 1 1 0 1 0 0 1

To better understand how different parity groupings overlap and how flipping a single bit affects the syndrome, you can experiment with the interactive Hamming Code visualizer below.

json

```
{"component": "LlmGeneratedComponent", "props": {"height": "700px", "prompt": "Objective: Build an interactive Hamming Code Error Correction visualizer that dynamically calculates syndromes based on user input.\n\nData State: \nInitial Code = '111001001111'. \nParity Type = 'Odd'. \nIndexing = 'Right-to-Left'.\n\nStrategy: Form layout combined with a dynamic interactive visual bit array.\n\nInputs: \n1. A text input field for the 12-bit binary string.\n2. A dropdown or toggle for Parity Type (Even / Odd).\n3. A dropdown or toggle for Indexing Direction (Left-to-Right / Right-to-Left).\n\nBehavior: \n- Render the 12 bits as distinct, interactive blocks in a row. Each block should display its index number above it (respecting the chosen Indexing Direction) and its bit value (0 or 1) inside.\n- Allow the user to click any bit block to instantly flip its value between 0 and 1.\n- Below the main bit array, display four distinct rows representing the parity checks (P1, P2, P4, P8). For each row, visually highlight or connect the specific bit blocks that are included in that parity group.\n- Dynamically calculate the syndrome for each row based on the selected Parity Type. If a row's parity check fails, mark it distinctly.\n- Display the final calculated Syndrome as a binary word and its decimal equivalent.\n- Visually highlight the specific bit block in the main array that corresponds to the calculated error position.", "id": "im_b3df9444e7fbdf8d"}}
```

You Asked 

(d) Compute the bit stream transmitted using Hamming code with even parity for the message $M = 1101101$. Show the steps to detect and correct an error at the receiver's end if the third bit from the left is inverted during transmission. 6

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question asks you to implement a **Hamming Code** to secure a message for transmission and then demonstrate its **Error Detection and Correction** capabilities.

Hamming code is a block code capable of detecting and correcting single-bit errors. It does this by intelligently inserting redundant "parity bits" into the original message data at specific mathematical positions.

Here is a breakdown of the core concept, the general steps to solve these problems, and the specific solution to your question.

The Concept: Hamming Code & Parity

- **Parity Bits vs. Data Bits:** In a Hamming code string, bits located at positions representing powers of 2 (Positions 1, 2, 4, 8, 16, etc.) are designated as **Parity Bits** (P_1, P_2, P_4, P_8). All other positions (Positions 3, 5, 6, 7, 9, 10, 11, etc.) are populated by the actual **Data Bits** of the original message.
- **Parity Coverage:** Each parity bit is responsible for monitoring a specific alternating pattern of bits within the transmission stream.
 - P_1 checks positions: 1, 3, 5, 7, 9, 11...
 - P_2 checks positions: 2, 3, 6, 7, 10, 11...
 - P_4 checks positions: 4, 5, 6, 7, 12...
 - P_8 checks positions: 8, 9, 10, 11, 12...

- **Even Parity:** The problem specifies "even parity." This rule dictates that the total number of 1s within any given parity group (including the parity bit itself) must add up to an **even number**. The parity bit's value is set to whatever is necessary (0 or 1) to make that count even.

Steps to Solve

Part 1: Generating the Transmission Stream

1. **Determine Parity Bits (p):** Use the formula $2^p \geq m + p + 1$ (where m is the number of data bits) to find the minimum number of parity bits needed.
2. **Position the Bits:** Lay out the total number of bits. Assign the data bits sequentially into the non-power-of-2 positions.
3. **Calculate Parity Values:** Evaluate the bits in each parity bit's coverage group. Set the parity bit to 1 or 0 to satisfy the parity rule (Even or Odd).

Part 2: Error Detection and Correction

1. **Introduce the Error:** Flip the bit at the position specified by the problem.
2. **Calculate the Syndrome:** The receiver evaluates the same parity groups on the received message. For each group, if the parity rule passes, the syndrome bit (S_x) is 0. If it fails, the syndrome bit is 1.
3. **Find the Error Position:** Arrange the syndrome bits in reverse order (e.g., $S_8 S_4 S_2 S_1$) to form a binary word. Convert this binary word to decimal to reveal the exact position of the corrupted bit.
4. **Correct the Error:** Invert the bit at the identified position to restore the original transmitted stream.

Demonstration: Solving the Question

Given Data:

- Message (M): 1101101
- Number of Data Bits (m): 7
- Rule: Even Parity

Phase 1: Compute the Transmitted Bit Stream

1. **Determine the number of parity bits (p):** Using $2^p \geq 7 + p + 1 \Rightarrow 2^p \geq p + 8$
 - If $p = 3$, $2^3 \geq 3 + 8 \Rightarrow 8 \geq 11$ (False)
 - If $p = 4$, $2^4 \geq 4 + 8 \Rightarrow 16 \geq 12$ (True). We need 4 parity bits (P_1, P_2, P_4, P_8). The total bit stream will be $7 + 4 = 11$ bits long.
2. **Position the bits (Left-to-Right Indexing):**

Position	1	2	3	4	5	6	7	8	9	10	11
Type	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}
Value	?	?	1	?	1	0	1	?	1	0	1

3. Calculate Parity Bit Values (Even Parity):

- P_1 (checks 1, 3, 5, 7, 9, 11): Bits are [?], 1, 1, 1, 1, 1. There are five 1s (Odd). To make the total even, $P_1 = 1$.
- P_2 (checks 2, 3, 6, 7, 10, 11): Bits are [?], 1, 0, 1, 0, 1. There are three 1s (Odd). To make the total even, $P_2 = 1$.
- P_4 (checks 4, 5, 6, 7): Bits are [?], 1, 0, 1. There are two 1s (Even). To keep the total even, $P_4 = 0$.

- P_8 (checks 8, 9, 10, 11): Bits are [?], 1, 0, 1. There are two 1s (Even). To keep the total even, $P_8 = 0$.

Transmitted Bit Stream: 1 1 1 0 1 0 1 0 1 0 1

Phase 2: Detect and Correct the Error

1. Introduce the Error: The question states the "third bit from the left is inverted."

- Transmitted Stream: 1 1 1 0 1 0 1 0 1 0 1
- Inverting Position 3 (from 1 to 0):
- **Received Stream:** 1 1 0 0 1 0 1 0 1 0 1

2. Calculate the Syndrome at the Receiver: The receiver checks the groups for Even Parity. If parity fails, the syndrome bit is 1.

- Check S_1 (pos 1, 3, 5, 7, 9, 11): 1, 0, 1, 1, 1, 1. Total 1s = 5 (Odd). Parity Fails $\Rightarrow S_1 = 1$.
- Check S_2 (pos 2, 3, 6, 7, 10, 11): 1, 0, 0, 1, 0, 1. Total 1s = 3 (Odd). Parity Fails $\Rightarrow S_2 = 1$.
- Check S_4 (pos 4, 5, 6, 7): 0, 1, 0, 1. Total 1s = 2 (Even). Parity Passes $\Rightarrow S_4 = 0$.
- Check S_8 (pos 8, 9, 10, 11): 0, 1, 0, 1. Total 1s = 2 (Even). Parity Passes $\Rightarrow S_8 = 0$.

3. Identify the Error Position: Read the syndrome bits from highest to lowest ($S_8 S_4 S_2 S_1$):

- Binary Syndrome: 0011₂
- Convert to Decimal: 3

4. Correct the Error: The syndrome mathematically identifies that the error occurred exactly at **Position 3**. By inverting the bit at Position 3 in the received message (changing it from 0 back to 1), the original bit stream is perfectly restored.

To help visualize how the original message bits map into the wider Hamming code structure, and how flipping any bit recalculates the syndrome, you can interact with this generator below.

json

```
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Objective: Build an interactive Hamming Code Generator and Error Simulator.\n\nData State: \nInitial Message = '1101101'. \nParity Type = 'Even'. \nIndexing = 'Left-to-Right'. \n\nStrategy: Form layout combined with a dynamic interactive visual bit array. \n\nInputs: \n1. A text input field for the Data Message (e.g., up to 11 bits of data). \n2. A dropdown/toggle for Parity Type (Even / Odd). \n3. A 'Generate Code' button. \n\nBehavior: \n- UPON GENERATION: Calculate the required parity bits and map the data bits and parity bits into a single visual array of blocks. \n- In the array, clearly distinguish (visually) the Parity bit blocks from the Data bit blocks. Each block must show its index number above it (Left-to-Right) and its current bit value (0 or 1). \n- INTERACTIVITY: Allow the user to click any bit block in the array to instantly flip its value (simulating an error during transmission). \n- ERROR DETECTION PANEL: Below the array, display the four (or more) Parity Checks (P1, P2, P4, P8). Dynamically evaluate each check based on the CURRENT state of the visual array. \n- Display the Syndrome word dynamically (e.g., 'Syndrome: 0011 = Position 3'). \n- Strongly highlight the bit block in the main array that corresponds to the calculated Syndrome decimal value to show exactly where the system detects the error.",
    "id": "im_f789b7f68f0aa7d6"
  }
}
```

You Asked 

(b) Consider a network of six routers labelled A, B, C, D, E and F. The cost of the links between the routers is as follows :

(i) A-B : 4
(ii) A-E : 5
(iii) B-C : 2
(iv) B-F : 6
(v) C-D : 3
(vi) C-E : 1
(vii) D-F : 7
(viii) F-E : 8

The following information has just arrived at the router C :

- From B : (5, 0, 8, 12, 6, 2)
- From D : (16, 12, 6, 0, 9, 10)
- From E : (7, 6, 3, 9, 0, 4)

The measured delays to B, D and E are 6, 3 and 5 respectively. Using distance vector routing algorithm, give the new routing table for C specifying both the delay and the outgoing line to use.

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question is based on the concept of **Distance Vector Routing (DVR)**, a fundamental dynamic routing algorithm used in computer networks (such as in the Routing Information Protocol, or RIP).

Here is a breakdown of the concept, the steps to solve these problems, and a step-by-step demonstration using the values provided in your image.

The Concept: Distance Vector Routing

In a distance vector routing protocol, every router maintains a routing table (a "vector") that lists its best known distance to every other node in the network.

Because routers only know their immediate surroundings, they rely on their neighbors to learn about the rest of the network. Periodically, routers send copies of their routing tables to their direct neighbors. When a router receives these updates, it uses the **Bellman-Ford equation** to calculate if going *through* that neighbor provides a cheaper/faster path to a destination than its currently known path.

The Bellman-Ford equation for a router x calculating the shortest path to destination y via neighbor v is: $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$ Where $c(x, v)$ is the cost to reach the neighbor, and $D_v(y)$ is the neighbor's reported distance to the destination.

Steps to Solve Distance Vector Problems

To manually update a routing table based on neighbor updates, follow these steps:

- 1. Identify the Inputs:** List the router you are evaluating, its immediate neighbors, and the direct cost (or delay) to reach each of those neighbors.
- 2. Organize Neighbor Vectors:** Write down the routing arrays received from each neighbor, mapping them to the correct destination nodes (usually alphabetically: A, B, C, D...).
- 3. Calculate Potential Paths:** For every possible destination in the network, calculate the total cost to reach it via each of the available neighbors: $\text{Total Cost} = \text{Delay to Neighbor} + \text{Neighbor's advertised distance to destination}$

4. **Find the Minimum (The "Greedy" Choice):** Compare the calculated costs for each destination. Select the lowest cost.
 5. **Build the New Table:** Record the minimum cost and the specific neighbor (outgoing line) that provided that minimum route. Note that the distance from a router to itself is always 0.
-

Demonstration: Updating Router C's Table

Based on the image, we are calculating the new routing table for **Router C**. (Note: While the problem provides a list of static link costs at the top, it explicitly states to use the newly "measured delays" to B, D, and E for the actual routing update. This is a common exam trick to ensure you are using dynamic data rather than static topology).

Step 1: Identify Inputs for Router C

- **Neighbors:** B, D, and E.
- **Measured delays to neighbors:**
 - $C \rightarrow B = 6$
 - $C \rightarrow D = 3$
 - $C \rightarrow E = 5$

Step 2: Organize Received Vectors

Assuming standard alphabetical order for destinations (A, B, C, D, E, F):

- From B: [A:5, B:0, C:8, D:12, E:6, F:2]
- From D: [A:16, B:12, C:6, D:0, E:9, F:10]
- From E: [A:7, B:6, C:3, D:9, E:0, F:4]

Step 3 & 4: Calculate Paths and Find Minimums

- **To Destination A:**
 - Via B: $6 + 5 = 11$
 - Via D: $3 + 16 = 19$
 - Via E: $5 + 7 = 12$
 - **Minimum:** 11 (via B)
- **To Destination B:**
 - Via B: $6 + 0 = 6$
 - Via D: $3 + 12 = 15$
 - Via E: $5 + 6 = 11$
 - **Minimum:** 6 (via B)
- **To Destination C:**
 - Router C to itself is always 0.
- **To Destination D:**
 - Via B: $6 + 12 = 18$
 - Via D: $3 + 0 = 3$
 - Via E: $5 + 9 = 14$
 - **Minimum:** 3 (via D)
- **To Destination E:**

- Via B: $6 + 6 = 12$
 - Via D: $3 + 9 = 12$
 - Via E: $5 + 0 = 5$
 - **Minimum:** 5 (via E)
- **To Destination F:**
 - Via B: $6 + 2 = 8$
 - Via D: $3 + 10 = 13$
 - Via E: $5 + 4 = 9$
 - **Minimum:** 8 (via B)

Step 5: Build the Final Routing Table

Here is the final required output for Router C:

Destination Node	Delay (Cost)	Outgoing Line
A	11	B
B	6	B
C	0	Local (-)
D	3	D
E	5	E
F	8	B

To help visualize how the Bellman-Ford equation evaluates multiple neighbor paths to find the most efficient route, you can explore the interactive distance vector calculator below.

```
json
{
  "component": "LlmGeneratedComponent",
  "props": {
    "height": "700px",
    "prompt": "Objective: Build an interactive Distance Vector Routing calculator to visualize the Bellman-Ford path selection process.\n\nData State: \nCurrent Node: C\nDestinations: A, B, C, D, E, F\nNeighbors: B, D, E\nCost to Neighbors: C->B (6), C->D (3), C->E (5)\nVectors:\nB: [5, 0, 8, 12, 6, 2]\nD: [16, 12, 6, 0, 9, 10]\nE: [7, 6, 3, 9, 0, 4]\n\nStrategy: Form Layout combined with dynamic data visualization.\n\nInputs:\n1. A dropdown menu to 'Select Destination Node' (A, B, D, E, F).\n2. Input fields to adjust the 'Delay to Neighbor' for B, D, and E (defaulting to 6, 3, 5).\n\nBehavior:\n- Display a summary of the received vectors in a static reference table at the top.\n- When a user selects a Destination Node from the dropdown, dynamically render three calculation panels below it, representing the path via neighbor B, via D, and via E.\n- Inside each panel, display the Bellman-Ford calculation: [Current Delay to Neighbor] + [Neighbor's Vector to Destination] = [Total Cost].\n- Distinctly highlight the panel that results in the lowest Total Cost (the chosen outgoing line).\n- At the bottom, maintain a dynamic 'Router C Final Routing Table' that automatically updates its values if the user changes the 'Delay to Neighbor' input fields.",
    "id": "im_e5df2b8c04dc3c7b"
  }
}
```

- (c) Given the IP address **200.1.2.30**, an organization needs to create two subnets :
- (i) Find the class of the given IP address using classful addressing.
- (ii) Determine the starting IP address and the last IP address of each subnet.
- (iii) Give the subnet mask for the given IP address.
- (iv) How many hosts can be on each subnet ?

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question is based on the concept of **Classful IP Addressing** and **Fixed-Length Subnet Masking (FLSM)**.

When an organization is given a block of IP addresses, it often needs to divide that large, single network into multiple smaller, independent networks (subnets) for security, organization, or traffic management. In this problem, because there are no specific host size requirements given for each subnet, we assume they need to be divided into equal-sized halves. We achieve this by "borrowing" bits from the host portion of the IP address and giving them to the network portion.

Here is a breakdown of the steps to solve these problems and the demonstration using your specific question.

Steps to Solve Classful Subnetting Problems

- 1. Identify the Class and Base Network:** Look at the first octet (number) of the IP address.
 - **Class A** (1–126): Default mask is **255.0.0.0**
 - **Class B** (128–191): Default mask is **255.255.0.0**
 - **Class C** (192–223): Default mask is **255.255.255.0**
- 2. Determine Borrowed Bits (n):** Use the formula $2^n \geq \text{Required Subnets}$ to find out how many host bits you must borrow to create the needed number of networks.
- 3. Determine Remaining Host Bits (h):** Subtract the borrowed bits from the total default host bits.
- 4. Calculate the New Subnet Mask:** Convert the network bits (including the newly borrowed ones) into 1s and the remaining host bits into 0s to get the binary mask, then convert it to decimal format.
- 5. Calculate Subnet Capacity:** * Total IPs per subnet = 2^h
 - Usable Hosts per subnet = $2^h - 2$ (Subtracting the Network ID and Broadcast ID).
- 6. Map the Subnet Ranges:** Starting from the base network address, add the total IPs per subnet (2^h) repeatedly to find the boundaries of each new subnet.

Demonstration: Solving the Question

Given Data: IP Address is **200.1.2.30** . Requirement is to create **2 subnets**.

(i) Find the class of the given IP address using classful addressing.

The first octet of the given IP is **200**. Because 200 falls within the range of 192 to 223, this is a **Class C** address.

- **Base Network:** **200.1.2.0**
- **Default Mask:** **255.255.255.0** (which gives us 8 host bits in the final octet).

(iii) Give the subnet mask for the given IP address.

Note: I am solving part (iii) before (ii) because you need the mask to determine the ranges. To create 2 subnets, we need to borrow bits from the 8 available host bits.

- Using the formula: $2^n \geq 2$.
- Therefore, $n = 1$. We must borrow **1 bit**.
- The default Class C mask in binary is: 11111111.11111111.11111111.00000000
- By borrowing 1 bit, the new mask becomes: 11111111.11111111.11111111.10000000
- Converting that last octet (10000000) to decimal gives us 128.
- **Answer:** The new subnet mask is **255.255.255.128**.

(iv) How many hosts can be on each subnet?

- We originally had 8 host bits and borrowed 1, leaving $h = 7$ bits for hosts.
- Using the usable host formula: $2^7 - 2 = 128 - 2$.
- **Answer:** There can be **126 usable hosts** on each subnet.

(ii) Determine the starting IP address and the last IP address of each subnet.

The "block size" (total IPs per subnet) is $2^7 = 128$. We divide the 256 available IPs in the last octet into blocks of 128. (Note: In networking, "starting and last IP" typically refers to the assignable host range, excluding the Network and Broadcast addresses).

Subnet 1:

- Network Address: 200.1.2.0
- **Starting IP: 200.1.2.1**
- **Last IP: 200.1.2.126**
- Broadcast Address: 200.1.2.127

Subnet 2:

- Network Address: 200.1.2.128
- **Starting IP: 200.1.2.129**
- **Last IP: 200.1.2.254**
- Broadcast Address: 200.1.2.255

To explore how manipulating the number of required equal-sized subnets impacts the borrowed bits, subnet mask, and available hosts for any base IP, you can use the interactive subnet calculator below.

```
json
{"component": "LlmGeneratedComponent", "props": {"height": "600px", "prompt": "Objective: Build an interactive Fixed-Length Subnet Masking (FLSM) calculator to demonstrate classful IP division.\n\nData State: \nBase IP: '200.1.2.0'.\nRequired Subnets: 2.\n\nStrategy: Form Layout combined with a dynamic results table.\n\nInputs: \n1. A text input field for 'Base IP Address' (defaulting to the Data State).\n2. A number input field for 'Number of Required Subnets' (defaulting to 2).\n3. A 'Calculate Subnets' button.\n\nBehavior: \n- UPON CALCULATION: First, determine the Address Class (A, B, or C) based on the first octet of the Base IP. \n- Calculate the required borrowed bits (n), the remaining host bits (h), and the new Subnet Mask.\n- Display a summary section showing: IP Class, Default Mask, New Subnet Mask, Bits Borrowed, and Usable Hosts per Subnet.\n- Below the summary, generate a data table that lists every created subnet (up to a reasonable display limit if the user requests hundreds). For each subnet row, display: Subnet"}}
```

Number, Network IP, First Usable IP, Last Usable IP, and Broadcast IP. \n- Ensure the table dynamically updates its ranges based on the calculated block size (2^h).", "id": "im_c7752902f3f3c832"}}}

You Asked 

(d) Eight 1-Kbps connections are multiplexed together using TDM. Each unit is 1-bit. Find :

- The duration of 1 bit before multiplexing
- The transmission rate of the link
- The duration of a time slot
- The duration of a frame.

explain the concept used in this question. define the steps to solve these types of question. demonstrate an example with the help of this question.

Gemini 3.1 Pro Extended

This question is based on the concept of **Time Division Multiplexing (TDM)**. TDM is a digital networking technique used to transmit multiple signals simultaneously over a single communication channel by dividing the channel's bandwidth into distinct, alternating time slots.

This concept is foundational to modern telecommunications, helping maximize the efficiency of high-bandwidth cables by preventing them from sitting idle while waiting for slow data streams.

Here is a breakdown of the core concept, the steps to solve any problem like this, and the specific solution to your question.

The Concept: Time Division Multiplexing (TDM)

Imagine a single, high-speed highway (the multiplexed link) that needs to carry cars from several different slow-moving local roads (the connections). TDM works like a very fast traffic cop.

- The Multiplexer:** The cop takes one car (a "unit" or "bit") from Road 1, then one from Road 2, then Road 3, and puts them onto the highway one by one.
- Time Slots:** The specific fraction of time allocated on the highway for a single car from a specific road is called a *Time Slot*.
- Frames:** Once the cop has taken exactly one car from every road, that complete cycle of cars on the highway is called a *Frame*. Therefore, one Frame contains exactly one Time Slot from each connection.
- Synchronization:** Because the highway is carrying traffic for everyone, it must travel much faster than the local roads to keep up. If there are 8 local roads, the highway must operate 8 times faster.

Steps to Solve TDM Problems

To solve standard TDM calculation problems, follow these mathematical steps:

- Identify Inputs:** Note the number of connections (N), the data rate of each connection (R_{in}), and the unit size being multiplexed (e.g., 1 bit, 1 byte).
- Calculate Input Bit Duration:** This is the time it takes for one input line to generate a single bit.
$$\text{Input Duration} = \frac{1}{R_{in}}$$

3. **Calculate Link Transmission Rate:** The total capacity of the multiplexed output link (R_{out}). Assuming no overhead or synchronization bits are mentioned: $R_{out} = N \times R_{in}$
4. **Calculate Time Slot Duration:** The time taken to transmit that single unit (e.g., 1 bit) on the *fast*, multiplexed link.
$$\text{Time Slot} = \frac{1}{R_{out}} \quad \text{or} \quad \frac{\text{Input Duration}}{N}$$
5. **Calculate Frame Duration:** The time it takes to transmit one entire frame. Since a frame consists of N time slots, you multiply the time slot duration by N . (Note: In basic TDM, the frame duration is always exactly equal to the Input Bit Duration!)
-

Demonstration: Solving Your Question

Given Data:

- Number of connections (N) = 8
- Input rate (R_{in}) = 1 Kbps = 1000 bps
- Unit size = 1 bit

(i) The duration of 1 bit before multiplexing

This is the time it takes for one of the original 1-Kbps connections to produce a single bit.

- $\text{Duration} = \frac{1}{1000 \text{ bps}}$
- **Answer:** 0.001 seconds (or 1 ms)

(ii) The transmission rate of the link

Since 8 connections are combining their traffic, the main link must be fast enough to carry all of them simultaneously.

- $\text{Rate} = 8 \times 1 \text{ Kbps}$
- **Answer:** 8 Kbps (or 8000 bps)

(iii) The duration of a time slot

A time slot is the time it takes the multiplexed link to transmit a single 1-bit unit. Because the link operates at 8000 bits per second:

- $\text{Time Slot} = \frac{1}{8000 \text{ bps}}$
- *Calculation:* $1 \div 8000 = 0.000125$ seconds.
- **Answer:** 125 μs (microseconds) or 0.125 ms

(iv) The duration of a frame

A complete frame contains one unit (1 time slot) from each of the 8 connections.

- $\text{Frame Duration} = 8 \times \text{Time Slot}$
- *Calculation:* $8 \times 125 \mu\text{s} = 1000 \mu\text{s}$.
- **Answer:** 1 ms (Notice how this perfectly matches the original duration of 1 bit before multiplexing).

To help you visualize how the multiplexer pulls individual units from slower streams and packs them into high-speed frames, you can interact with this TDM visualizer below.

json

```
{"component": "LlmGeneratedComponent", "props": {"height": "700px", "prompt": "Objective: Build an interactive visualizer for Time Division Multiplexing (TDM) to demonstrate how frames and time slots are constructed.\n\nData State:"
```

```
\nInitial Connections (N) = 8.\nInitial Input Rate = 1 Kbps.\nUnit Size = 1 bit.\n\nStrategy: Form Layout combined with dynamic animation and a data table.\n\nInputs: \n1. A slider to adjust the 'Number of Connections (N)' from 2 to 8.\n2. A number input to adjust the 'Input Rate (Kbps)'. \n\nBehavior: \n- Render an animation area showing N distinct 'Input Streams' on the left (represented by differently colored bit blocks moving slowly). \n- Show a 'Multiplexer' funnel in the center that takes one colored block from each input stream in sequence.\n- On the right, animate the 'Multiplexed Link' moving much faster. The blocks on this link should be grouped into 'Frames', with each frame containing exactly N distinctly colored 'Time Slots' matching the inputs.\n- Below the animation, maintain a dynamic results table that updates instantly when inputs change. It must display:\n 1. Duration of 1 input bit (ms)\n 2. Transmission rate of link (Kbps)\n 3. Duration of a time slot ( $\mu$ s)\n 4. Duration of a frame (ms)\n- Ensure the calculations match the mathematical formulas for TDM.", "id": "im_a291f58e3ae4bf39"}}
```